
Uvod u distribuirano programiranje

— Problemi i rješenja —

Šta je problem?

- Velike količine podataka
- Datasetovi u mašinskom učenju veličine reda 100GB ili TB
- Ne može stati u radnu memoriju a često ni u fajl sistem jednog računara
- Potreban pristup koji može da radi bez obzira na količinu podataka
- Optimizacije igraju bitnu ulogu - Jako bitno koliko se izvršava naš program (da li je dan, sedmica ili mjesec)

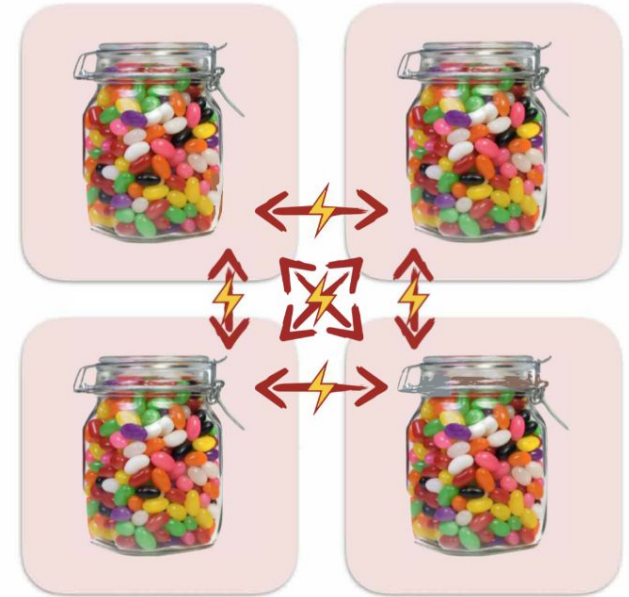
Paralelno vs distribuirano programiranje

- Osnovna ideja ista - podijeliti podatke u manje dijelove i kreirati više procesa - svaki proces obrađuje dio podataka
- Razlika - gdje se nalaze podaci:
 - paralelno - podaci particionisani u memoriji
 - distribuirano - podaci particionisani između više mašina
- Jako bitan problem - latency - brzina pristupa memoiji puno veća od brzine komunikacije u mreži
- Iste ideje iz paralelnog programiranja se mogu primijeniti u distribuiranom ali uz određena ograničenja

Shared memory:



Distributed:



L1 cache reference	0.5ns	
Branch mispredict	5ns	
L2 cache reference	7ns	
Mutex lock/unlock	25ns	
Main memory reference	100ns	
Compress 1K bytes with Zippy	3,000ns	= 3μs
Send 2K bytes over 1Gbps network	20,000ns	= 20μs
SSD random read	150,000ns	= 150μs
Read 1 MB sequentially from	250,000ns	= 250μs
Roundtrip within same datacenter	500,000ns	= 0.5ms
Read 1MB sequentially from SSD	1,000,000ns	= 1ms
Disk seek	10,000,000ns	= 10ms
Read 1MB sequentially from disk	20,000,000ns	= 20ms
Send packet US → Europe → US	150,000,000ns	= 150ms

Šta je Spark?

- Programski okvir za rad sa velikim količinama podata
- Industrijski standard
- Podržava veliki broj programskih jezika
- Optimizuje distribuirane operacije i omogućava rad i skaliranje na veliki broj mašina
- Posjeduje proširenja i biblioteke za rad sa Mašinskim učenjem

Spark RDD

- Resilient Distributed Dataset osnova Sparka
- Svi podaci su nepromjenljivi i operacije se ralizuju kao kompozicija funkcionalnih transformacija
- 2 tipa operacija - transformacije i akcije:
 - transformacije mijenjaju kolekciju (map, filter, ...) - lazy
 - akcije generišu rezultat (reduce, collect) - eager
- Lazy vs eager execution
- Spark optimizuje operacije na osnovu željenog rezultata

Problem - funkcionalno programiranje

- Nova paradigma, potpuno drugačija od OOP
- Funkcije građani prvog reda, problemi se rješavaju kompozicijom funkcija
- Potrebno vrijeme da se prilagodi - previše smo se navikli na mutacije (dodjele i setteri), indeksiranja i for petlje
- Ideje mogu pomoći da OOP kod bude lakši za održavanje i proširivanje - pisanje manjih funkcija, minimizacija broja bočnih efekata

Standardne operacije

```
abstract class RDD[T] {  
  def map[U](f: T => U): RDD[U] = ...  
  def flatMap[U](f: T => TraversableOnce[U]): RDD[U] = ...  
  def filter(f: T => Boolean): RDD[T] = ...  
  def reduce(f: (T, T) => T): T = ...  
}
```